

第七章 卷积神经网络与 自然语言处理

本课程目标



了解卷积神经网络的概念和特性



掌握使用CNN进行自然语言处理的基本过程



掌握CNN进行自然语言处理过程中的一些超参数

课程目录

Course catalogue

1/ 卷积神经网络概述

2/ 卷积神经网络与自然语言处理

3/ CNN在应用中的超参数选择



从图中找出猫

“猫咪检测器”

分割区域

为每个区域包含猫咪的可能性打分



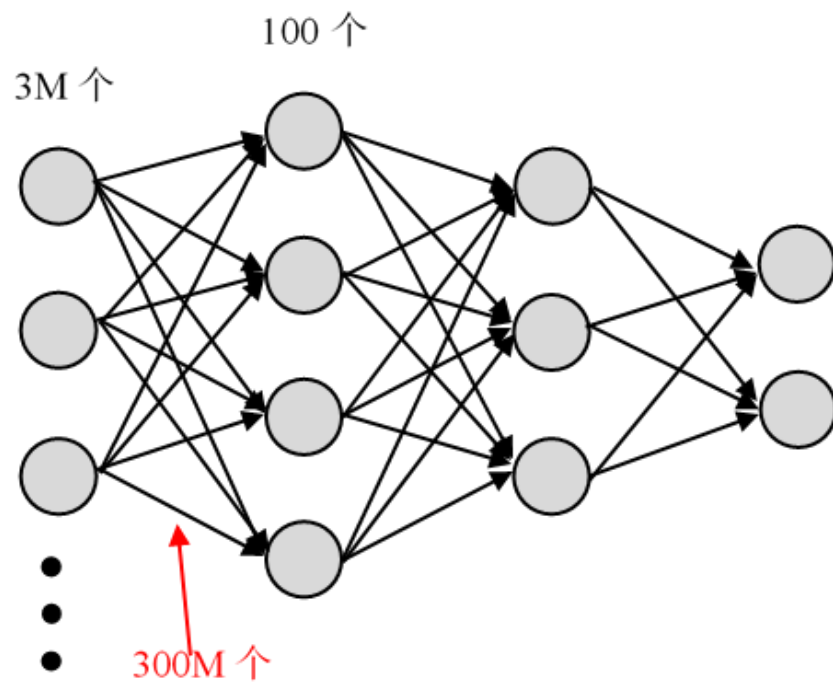
•**平移不变性 (translation invariance)**：不管检测对象出现在图像中的哪个位置，神经网络的前面几层应该对相同的图像区域具有相似的反应，即为“平移不变性”。

•**局部性 (locality)**：神经网络的前面几层应该只探索输入图像中的局部区域，而不过度在意图像中相隔较远区域的关系，这就是“局部性”原则。最终，可以聚合这些局部特征，以在整个图像级别进行预测。

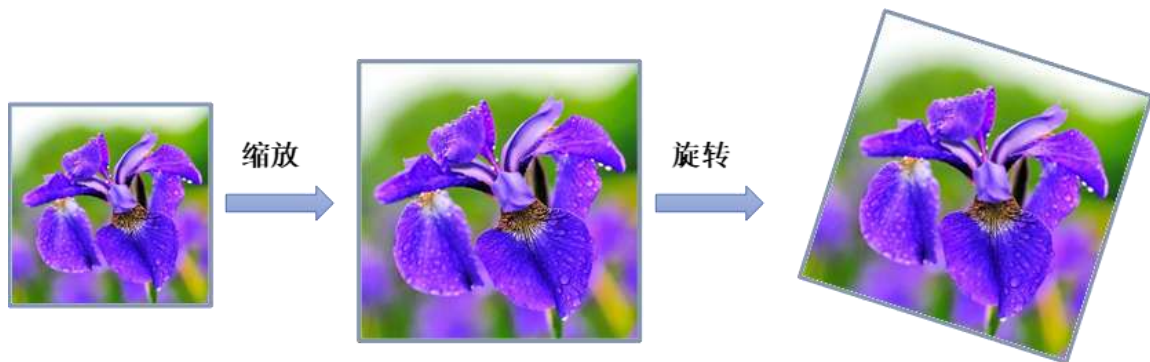
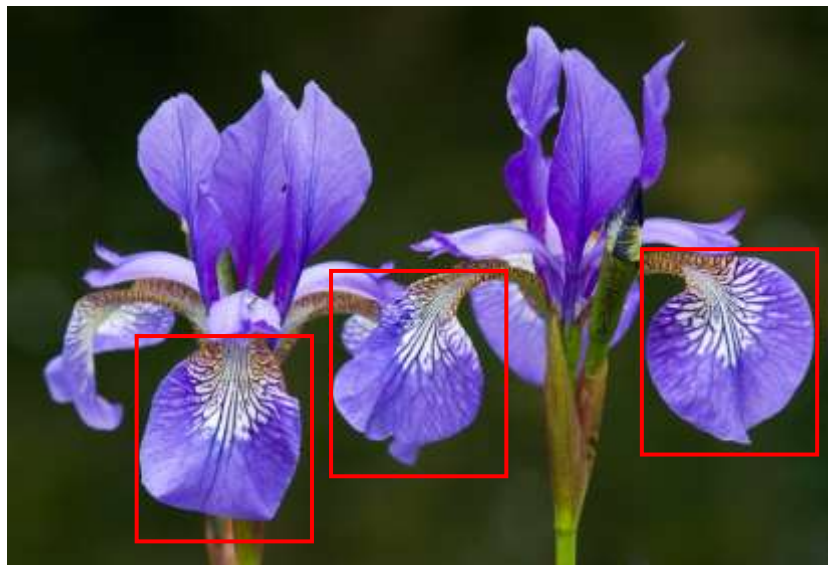
卷积神经网络概述

深层网络用于图像处理：参数过多

全连接网络，每个神经元到输入层都有连接，而每个连接都对应一个权重参数，因此，随着层数的增加，参数的规模也会急剧增加，这将会导致整个神经网络的训练效率非常低。

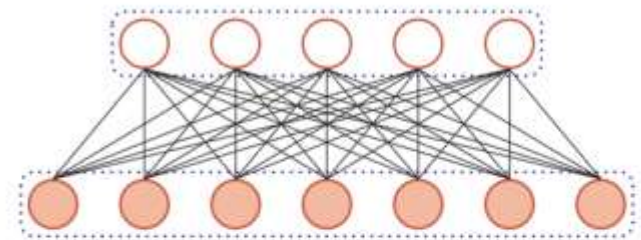


卷积神经网络概述



全连接前馈神经网络

▶ 权重矩阵的参数非常多



▶ 局部不变性特征

- ▶ 自然图像中的物体都具有局部不变性特征
 - ▶ 尺度缩放、平移、旋转等操作不影响其语义信息。
- ▶ 全连接前馈网络很难提取这些局部不变特征

卷积神经网络概述

什么是卷积 (Convolution)

卷积经常用在信号处理中，用于计算信号的延迟累积。

假设一个信号发生器每个时刻 t 产生一个信号 x_t ，其信息的衰减率为 w_k ，即在 $k-1$ 个时间步长后，信息为原来的 w_k 倍

假设 $w_1 = 1, w_2 = 1/2, w_3 = 1/4$

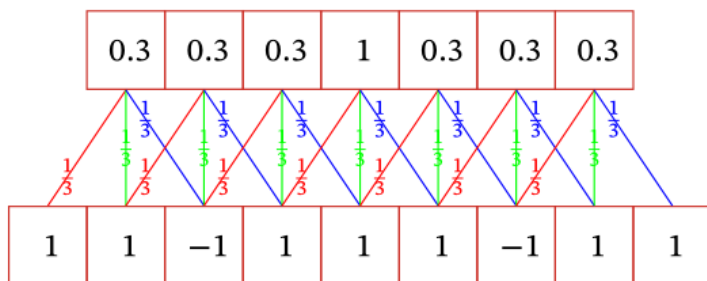
时刻 t 收到的信号 y_t 为当前时刻产生的信息和以前时刻延迟信息的叠加。

$$\begin{aligned} y_t &= 1 \times x_t + 1/2 \times x_{t-1} + 1/4 \times x_{t-2} \\ &= w_1 \times x_t + w_2 \times x_{t-1} + w_3 \times x_{t-2} \\ &= \sum_{k=1}^3 w_k \cdot x_{t-k+1}. \end{aligned}$$

滤波器 (filter) 或卷积核 (convolution kernel)

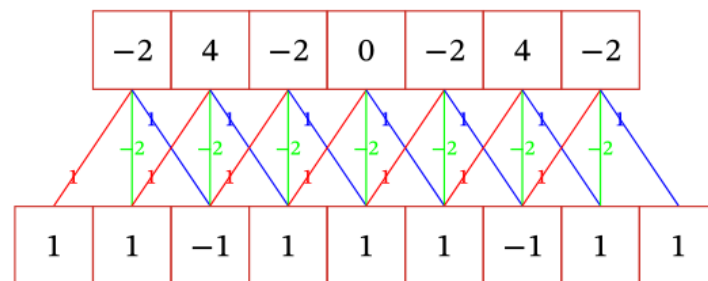
卷积

- 不同的滤波器来提取信号序列中的不同特征



(a) 滤波器 $[1/3, 1/3, 1/3]$

低频信息



(b) 滤波器 $[1, -2, 1]$

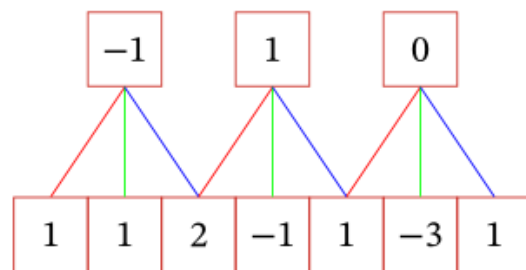
高频信息

$$y''(u) = y(u + 1) + y(u - 1) - 2y(u)$$

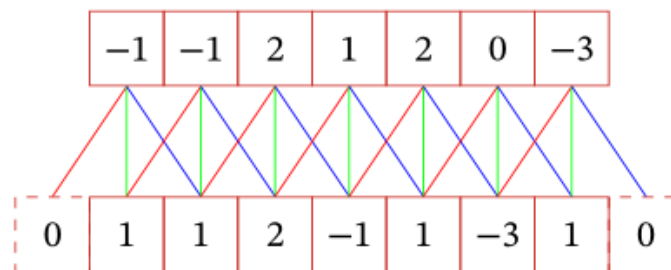
二阶微分

卷积扩展

- 引入滤波器的滑动步长 S 和零填充 P



(a) 步长 $S = 2$



(b) 零填充 $P = 1$

- 卷积的结果按输出长度不同可以分为三类：
 - 窄卷积：步长 $T = 1$ ，两端不补零 $P = 0$ ，卷积后输出长度为 $M - K + 1$
 - 宽卷积：步长 $T = 1$ ，两端补零 $P = K - 1$ ，卷积后输出长度 $M + K - 1$
 - 等宽卷积：步长 $T = 1$ ，两端补零 $P = (K - 1)/2$ ，卷积后输出长度 M

二维卷积

- 在图像处理中，图像是以二维矩阵的形式输入到神经网络中，因此我们需要二维卷积。

一个输入信息 $\mathbf{X}$ 和滤波器 $\mathbf{W}$ 的二维卷积定义为

$$\mathbf{Y} = \mathbf{W} * \mathbf{X},$$

$$y_{ij} = \sum_{u=1}^U \sum_{v=1}^V w_{uv} x_{i-u+1, j-v+1}.$$

1	1	1	1	1
-1	0	-3	0	1
2	1	1	-1	0
0	-1	1	2	1
1	2	1	1	1

$\times -1$

$\times 0$

$\times 0$

$\times 0$

$\times 0$

$\times 0$

$\times 0$

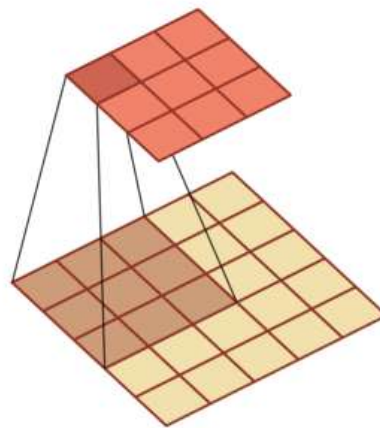
$\times 0$

$\times 1$

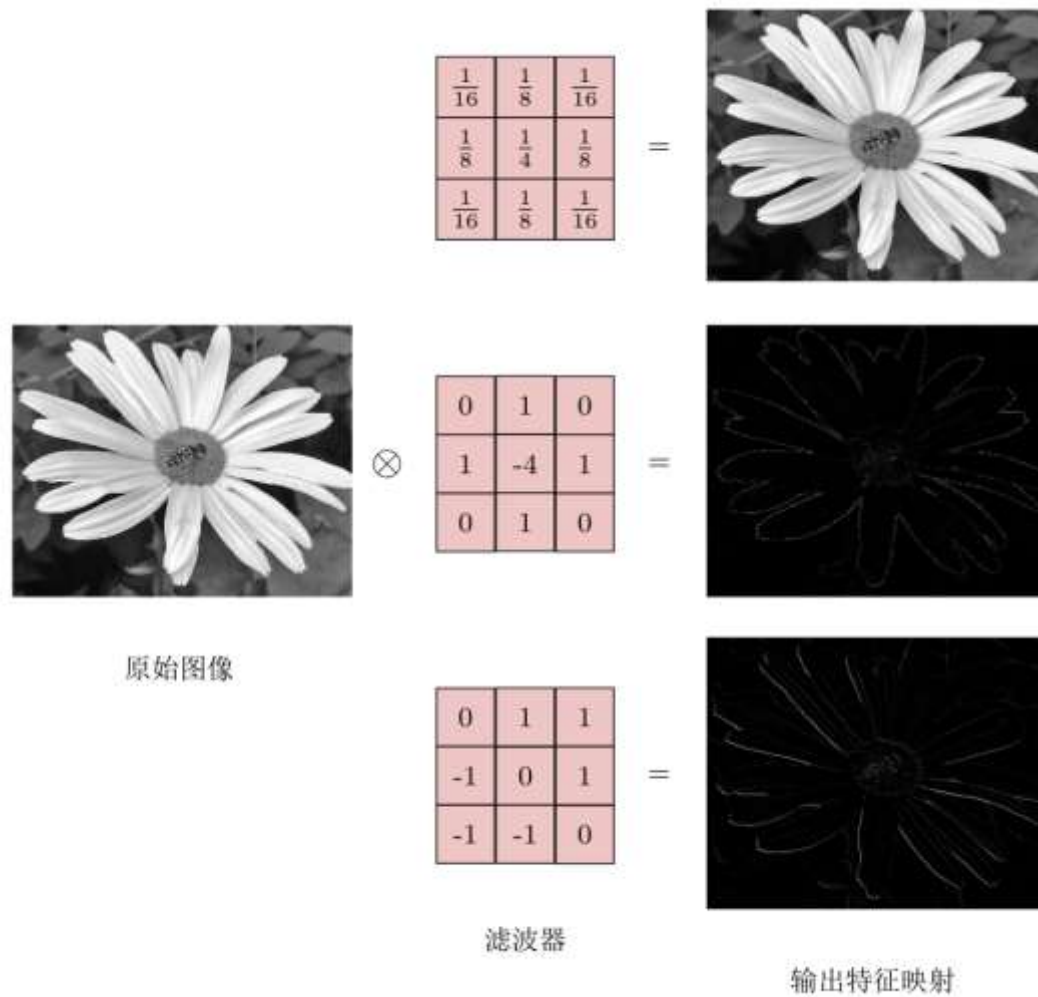
1	0	0
0	0	0
0	0	-1

$=$

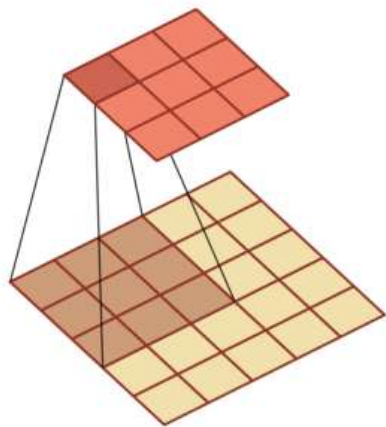
0	-2	-1
2	2	4
-1	0	0



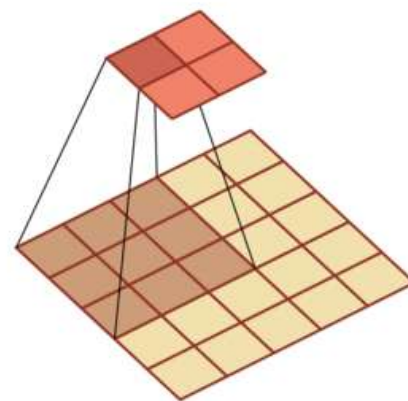
卷积作为特征提取器



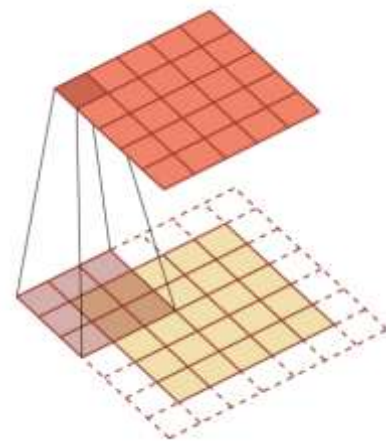
二维卷积



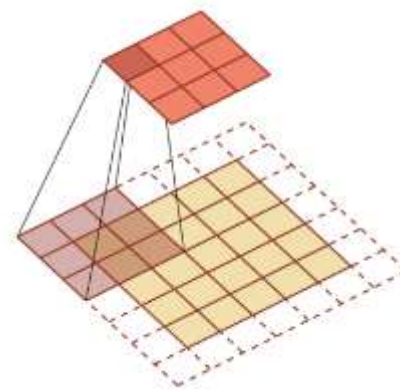
步长1，零填充0



步长2，零填充0



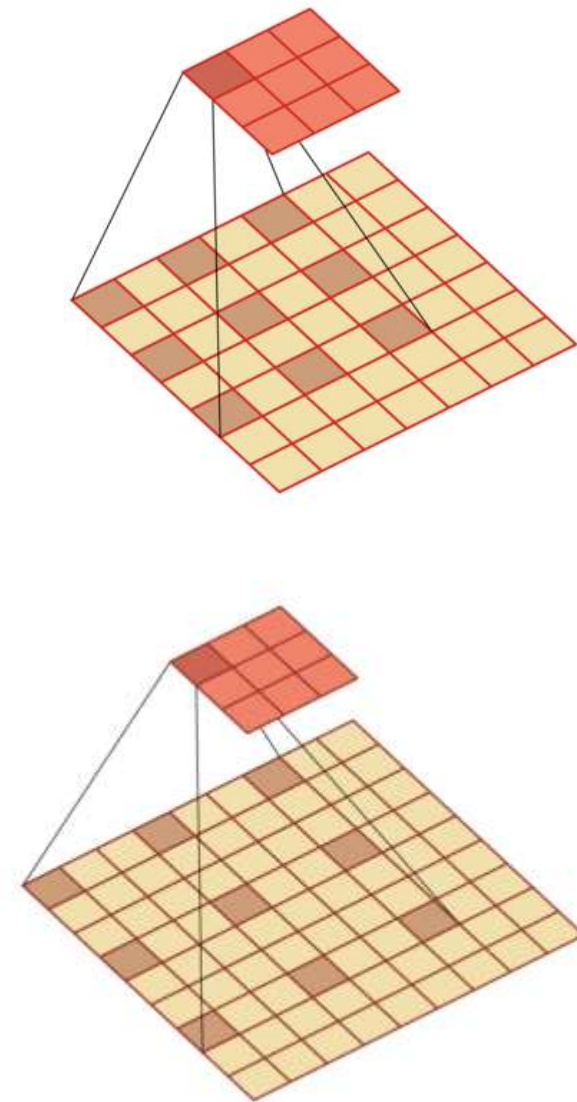
步长1，零填充1



步长2，零填充1

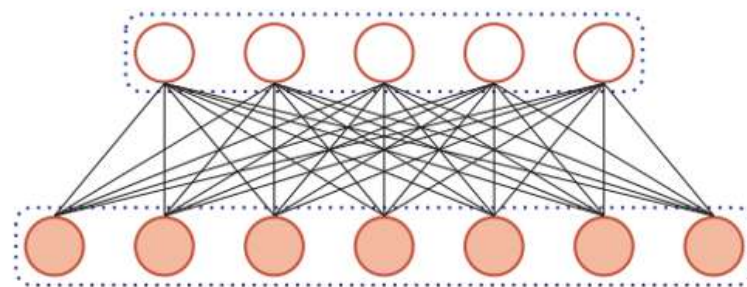
空洞卷积

- 如何增加输出单元的感受野
 - 增加卷积核的大小
 - 增加层数来实现
 - 在卷积之前进行汇聚操作
- 空洞卷积
 - 通过给卷积核插入“空洞”来变相地增加其大小。

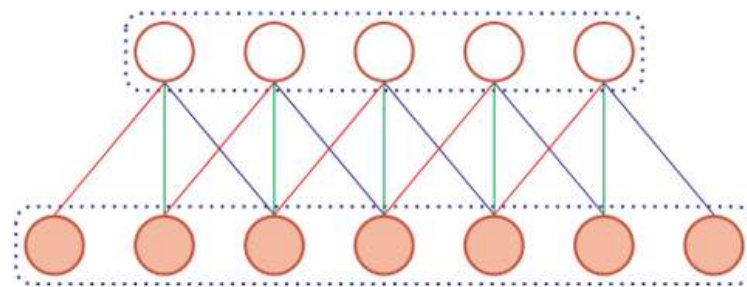


卷积神经网络概述

► 用卷积层代替全连接层



(a) 全连接层



(b) 卷积层

互相关 (cross-correlation)

- 计算卷积需要进行卷积核**翻转**。
- 卷积操作的目标：**提取特征**。

翻转是不必要的！

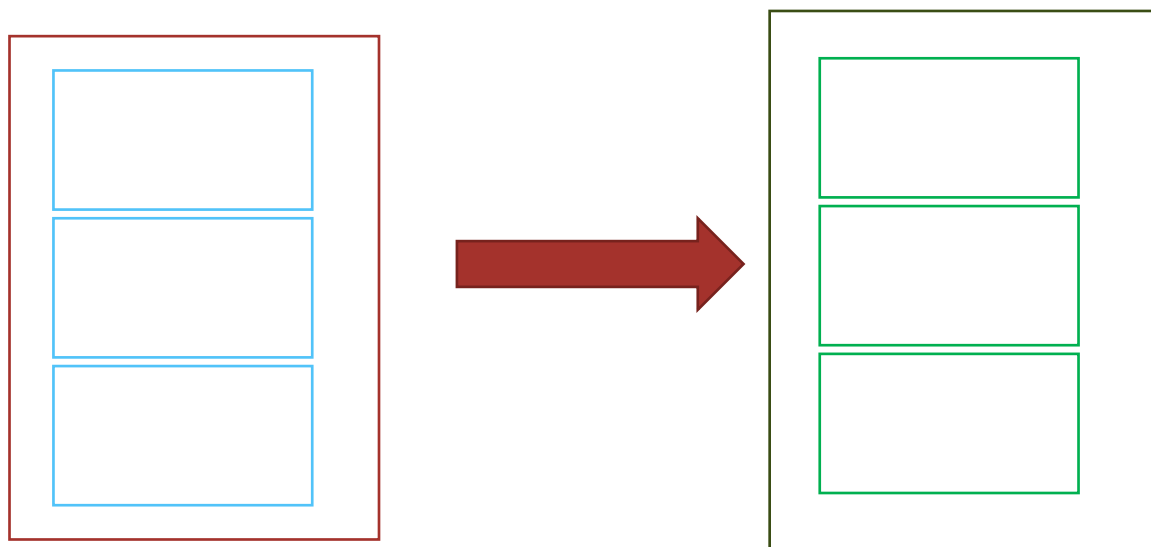
- 互相关

$$y_{ij} = \sum_{u=1}^m \sum_{v=1}^n w_{uv} \cdot x_{i+u-1, j+v-1}$$

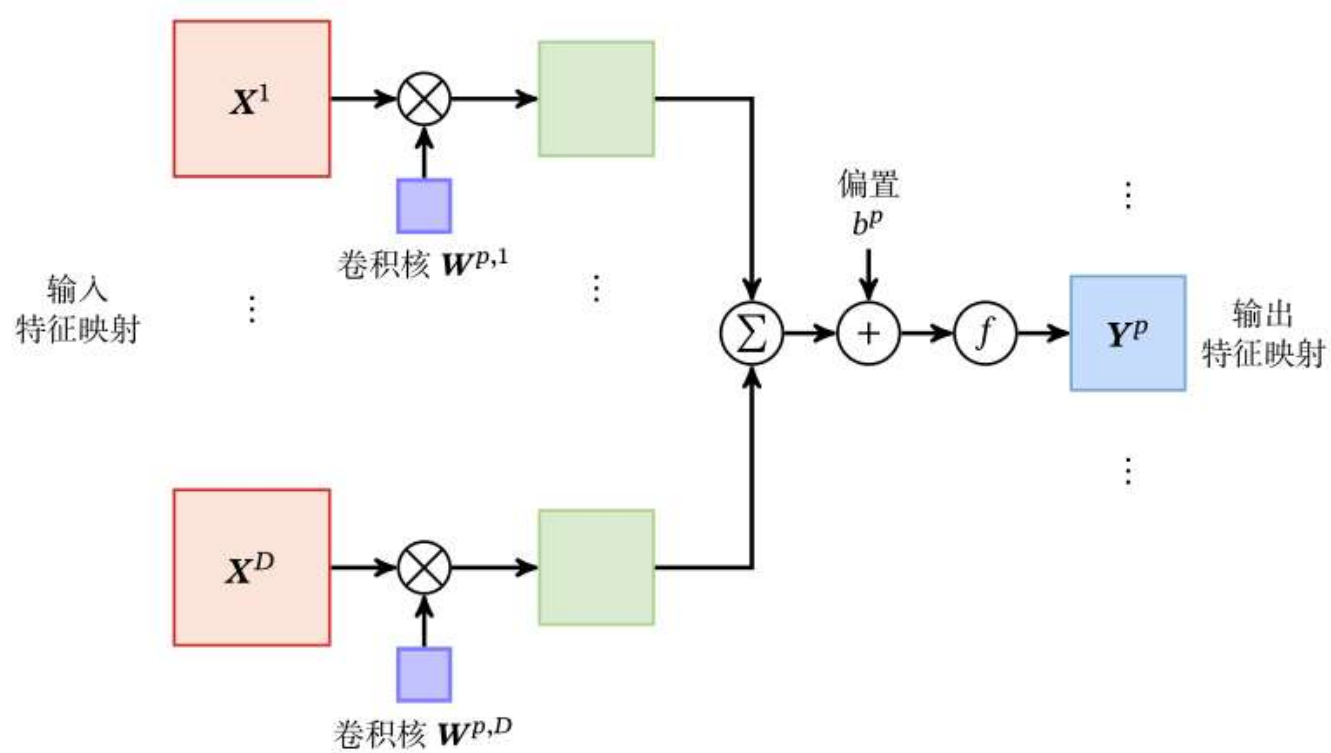
除非特别声明，卷积一般指“互相关”。

多个卷积核

- 特征映射 (Feature Map)：图像经过卷积后得到的特征。
 - 卷积核看成一个特征提取器
- 卷积层
 - 输入：D个特征映射 $M \times N \times D$
 - 输出：P个特征映射 $M' \times N' \times P$

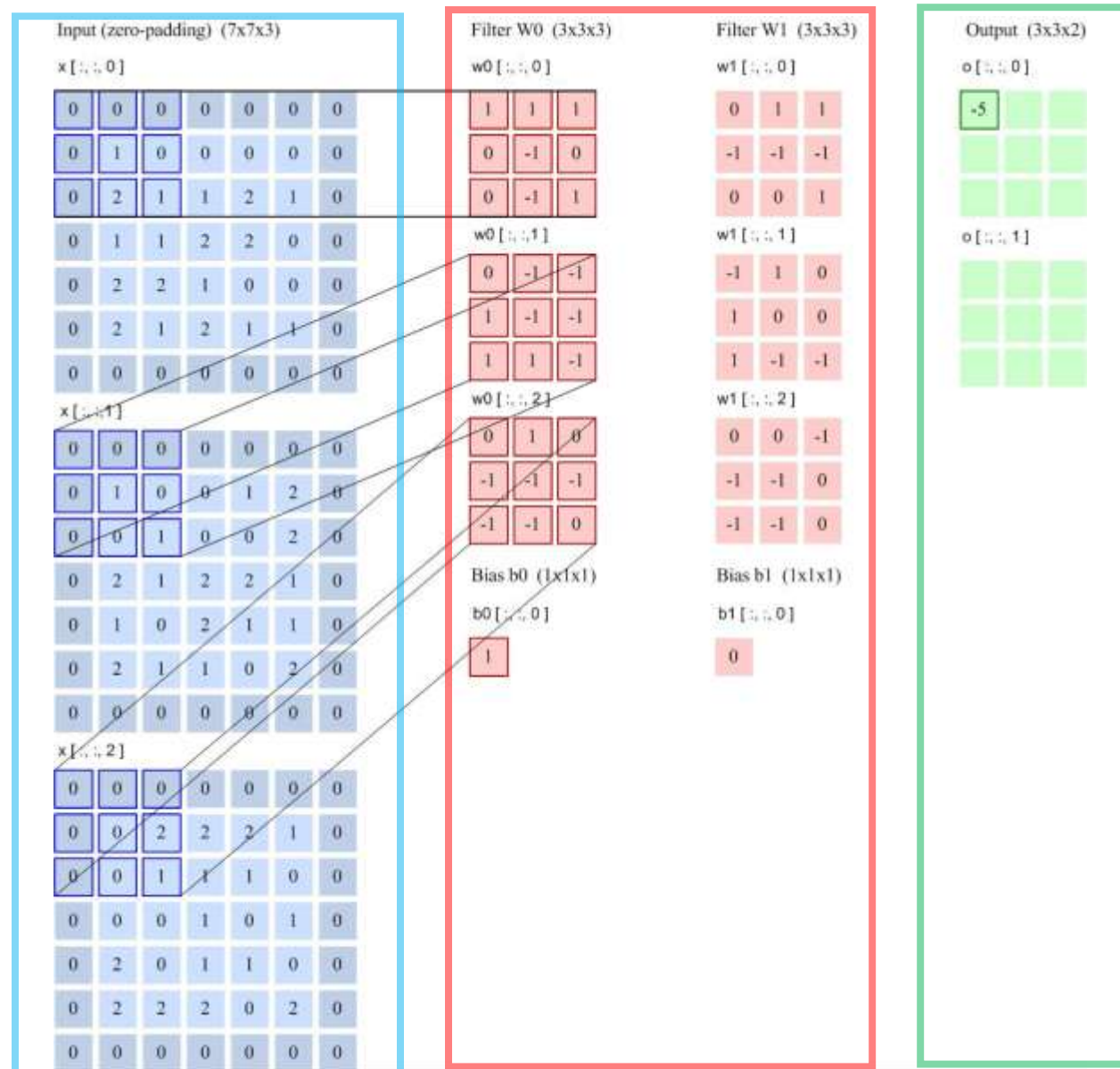


卷积层的映射关系



$$Z^p = W^p \otimes X + b^p = \sum_{d=1}^D W^{p,d} \otimes X^d + b^p,$$

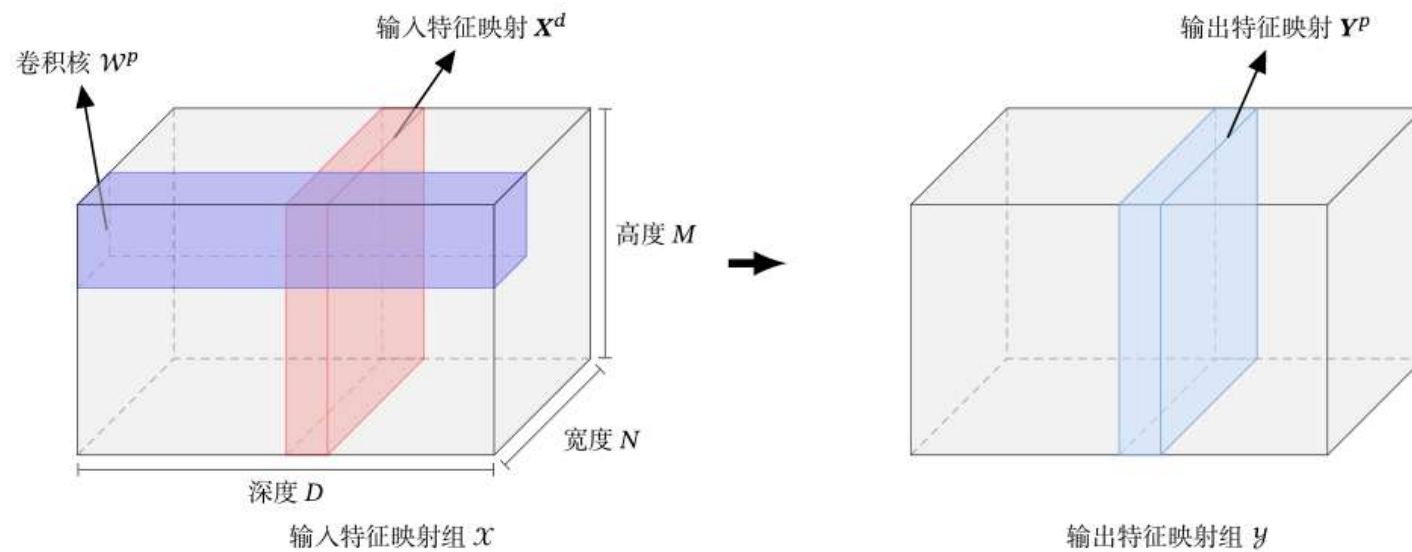
$$Y^p = f(Z^p).$$



步长2
filter 3*3
filter个数6
零填充 1

卷积层

- 典型的卷积层为3维结构

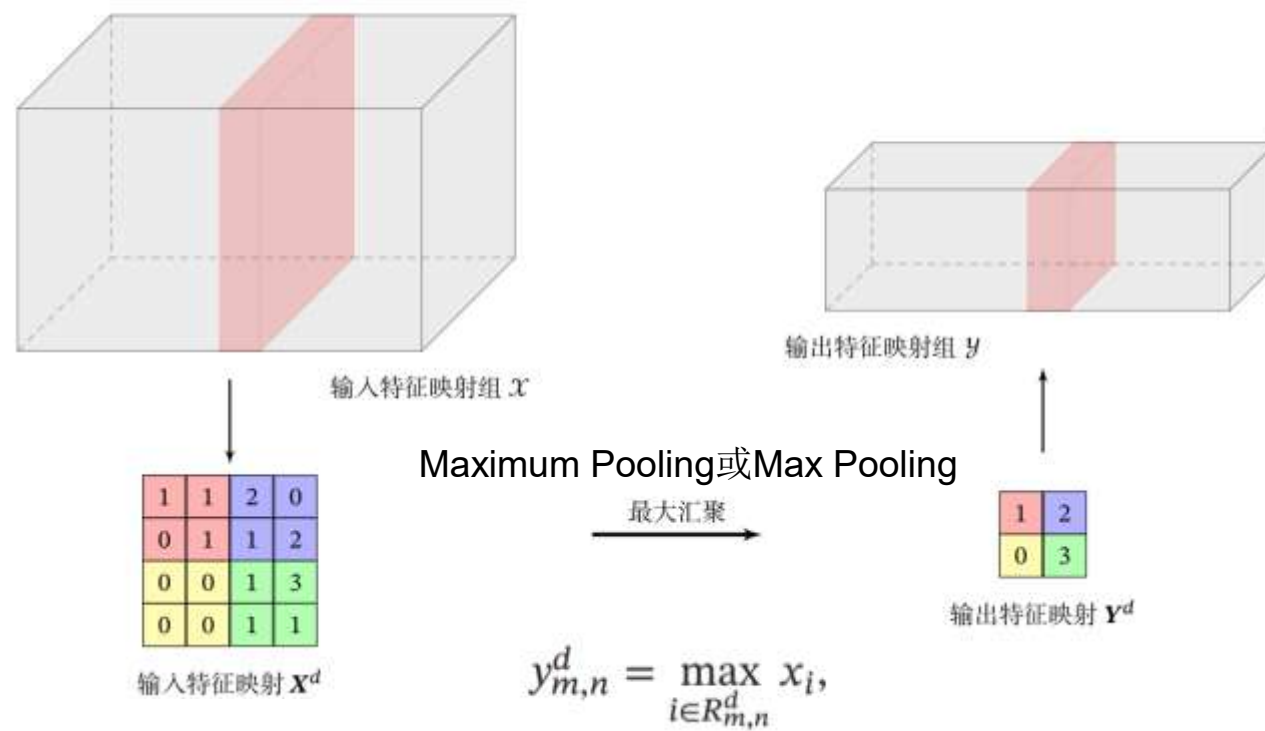


$$Z^p = W^p \otimes X + b^p = \sum_{d=1}^D W^{p,d} \otimes X^d + b^p,$$

$$Y^p = f(Z^p).$$

汇聚层

- 卷积层虽然可以显著减少连接的个数，但是每一个特征映射的神经元个数并没有显著减少。



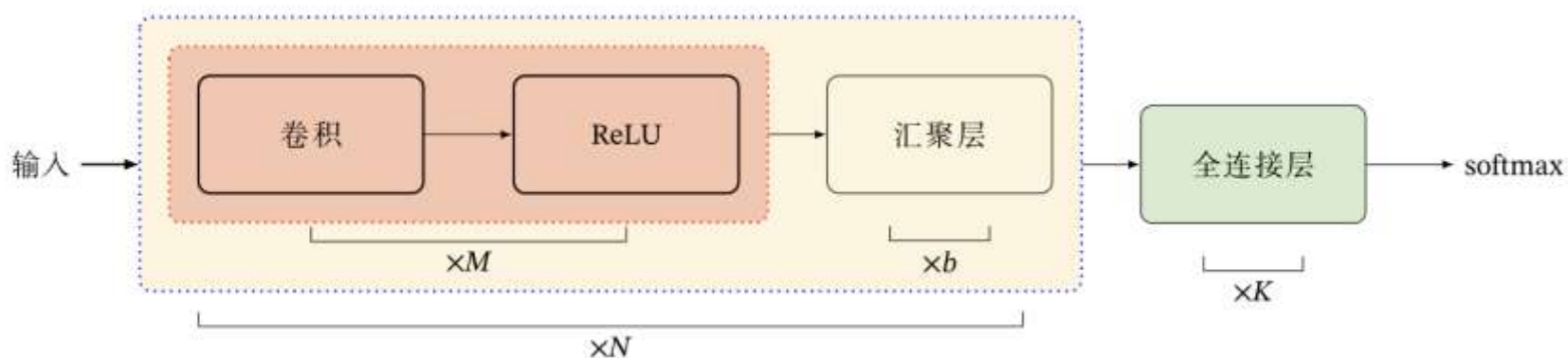
其他？

Mean Pooling

$$y_{m,n}^d = \frac{1}{|R_{m,n}^d|} \sum_{i \in R_{m,n}^d} x_i.$$

卷积网络结构

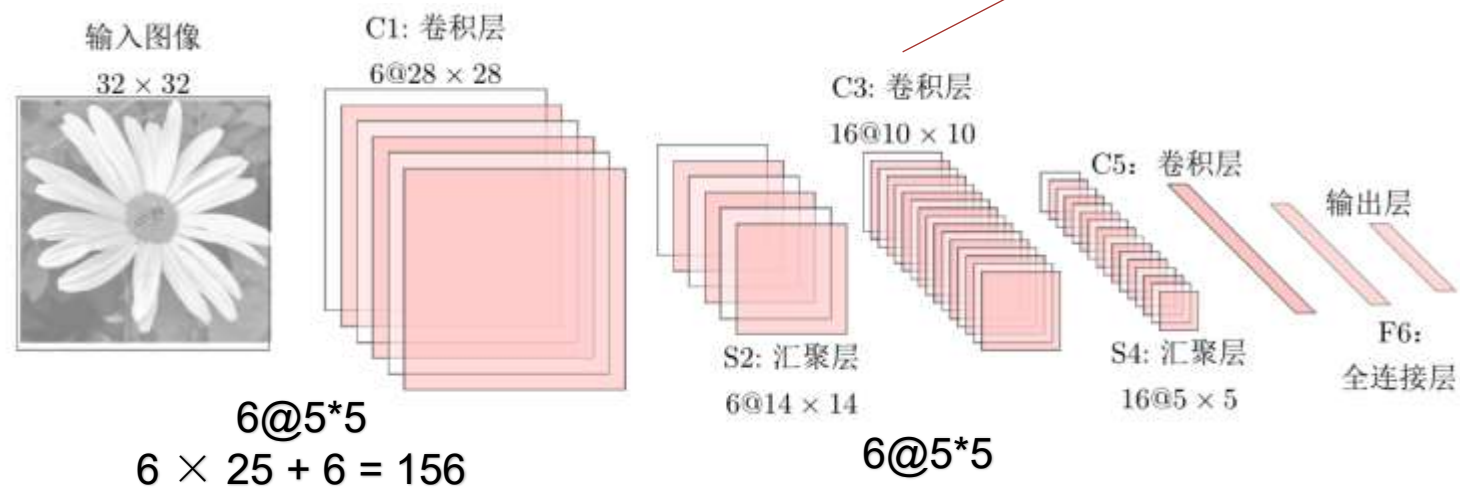
- 卷积网络是由卷积层、汇聚层、全连接层交叉堆叠而成。
- 典型结构



- 一个卷积块为连续 M 个卷积层和 b 个汇聚层 (M 通常设置为 $2 \sim 5$, b 为0或1)。一个卷积网络中可以堆叠 N 个连续的卷积块, 然后在接着 K 个全连接层 (N 的取值区间比较大, 比如 $1 \sim 100$ 或者更大; K 一般为 $0 \sim 2$)。

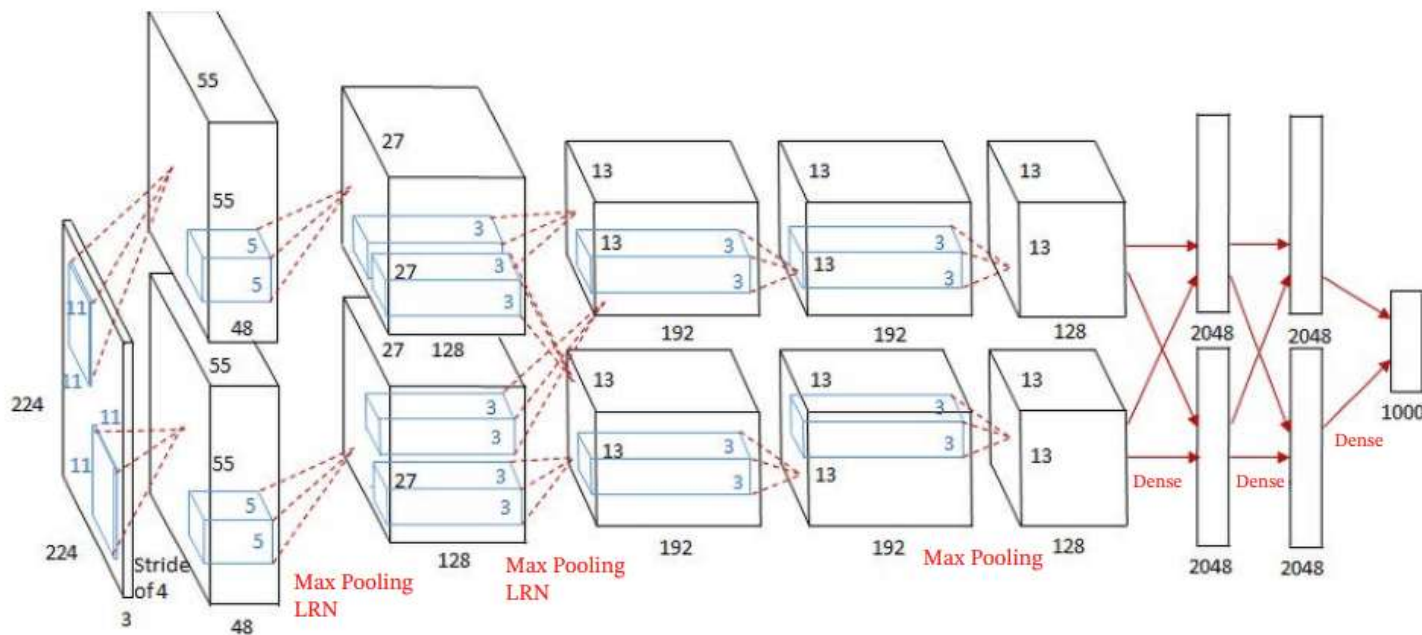
LeNet-5

- LeNet-5[LeCun et al., 1998] 是一个非常成功的神经网络模型。
 - 基于 LeNet-5 的手写数字识别系统在 90 年代被美国很多银行使用，用来识别支票上面的手写数字。
 - LeNet-5 共有 7 层。



AlexNet

- 2012 ILSVRC winner
 - (top 5 error of 16% compared to runner-up with 26% error)
 - AlexNet[Krizhevsky et al., 2012] 是第一个现代深度卷积神经网络模型
 - 首次使用了很多现代深度卷积网络的一些技术方法
 - 使用GPU进行并行训练，采用了ReLU作为非线性激活函数，使用Dropout防止过拟合，使用数据增强
 - 5个卷积层、3个汇聚层和3个全连接层



1. 卷积神经网络（**CNN**）在图像处理领域的应用已有数十年的历史。以下哪项不是**CNN**在图像处理中的重要应用或突破？

- A. 特征提取与图像分类
- B. 图像风格转换
- C. 三维图像重建
- D. 增强现实游戏

2. 图片分类中，**CNN**相对于全连接的**DNN**有什么优势？

（a）由于只是局部连接，同时高度复用weight，**CNN**比全连接**DNN**少很多参数，这使得它训练更快，减少了过拟合的风险，需要的训练集小得多。

（b）当**CNN**学习到检测某个特征的核（kernel）的时候，它可以在图片的任何位置检测到这个特征。而**DNN**在某个位置学习到某个特征，它只能在这个位置识别它。由于图片一般有很多重复的特征，**CNN**在分类等领域可以用更少的训练集比**DNN**泛化地好得多。

课程目录

Course catalogue

1/ 卷积神经网络概述

2/ 卷积神经网络与自然语言处理

3/ CNN在应用中的超参数选择



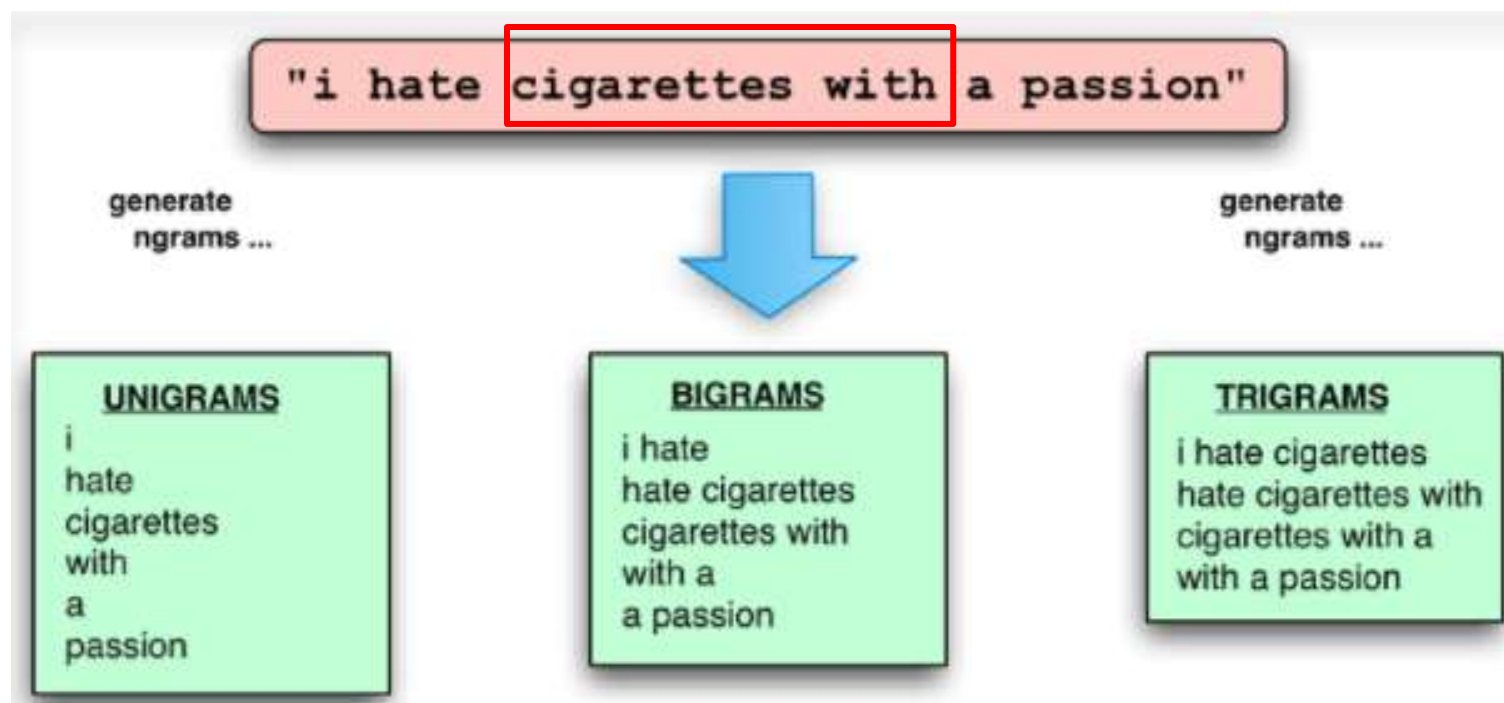
思考一下

- 1. 自然语言处理任务中有类似于图像的局部特征吗？
- 2. 句子的局部性是什么？
- 3. 举例说明自然语言处理任务中关键信息位置不固定的现象？



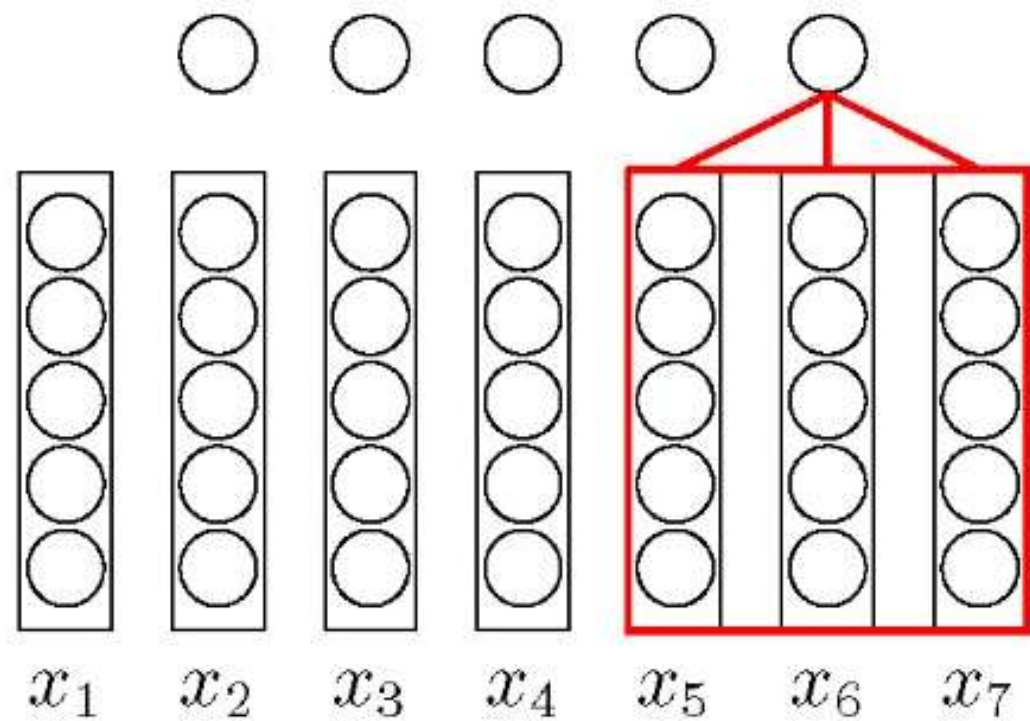
一个直接的想法是用一个稠密层提取局部特征

Ngram特征与卷积



如何用卷积操作来实现？

文本序列的卷积



例如，文本的1D卷积

Today	0.2	0.1	-0.3	0.4
is	0.5	0.2	-0.3	-0.1
a	-0.1	-0.3	-0.2	0.4
great	0.3	-0.3	0.1	0.1
day	0.2	0.1	0.3	0.4

k=3 的滤波

3	1	2	-3
-1	1	1	-1
-1	-2	3	4



t,i,a	0.1
i,a,g	?
a,g,d	?

P=1

$\emptyset$	0	0	0	0
Today	0.2	0.1	-0.3	0.4
is	0.5	0.2	-0.3	-0.1
A	-0.1	-0.3	-0.2	0.4
Great	0.3	-0.3	0.1	0.1
Day	0.2	-0.3	0.4	0.2
$\emptyset$	0	0	0	0



$\emptyset, t, i$	-0.6	0.2	1.4
t, i, a	-1.0	1.6	-1.0
i, a, g	-0.5	-0.1	0.8
a, g, d	-3.6	0.3	0.3
g, d, $\emptyset$	-0.2	0.1	1.2

通道为3的滤波

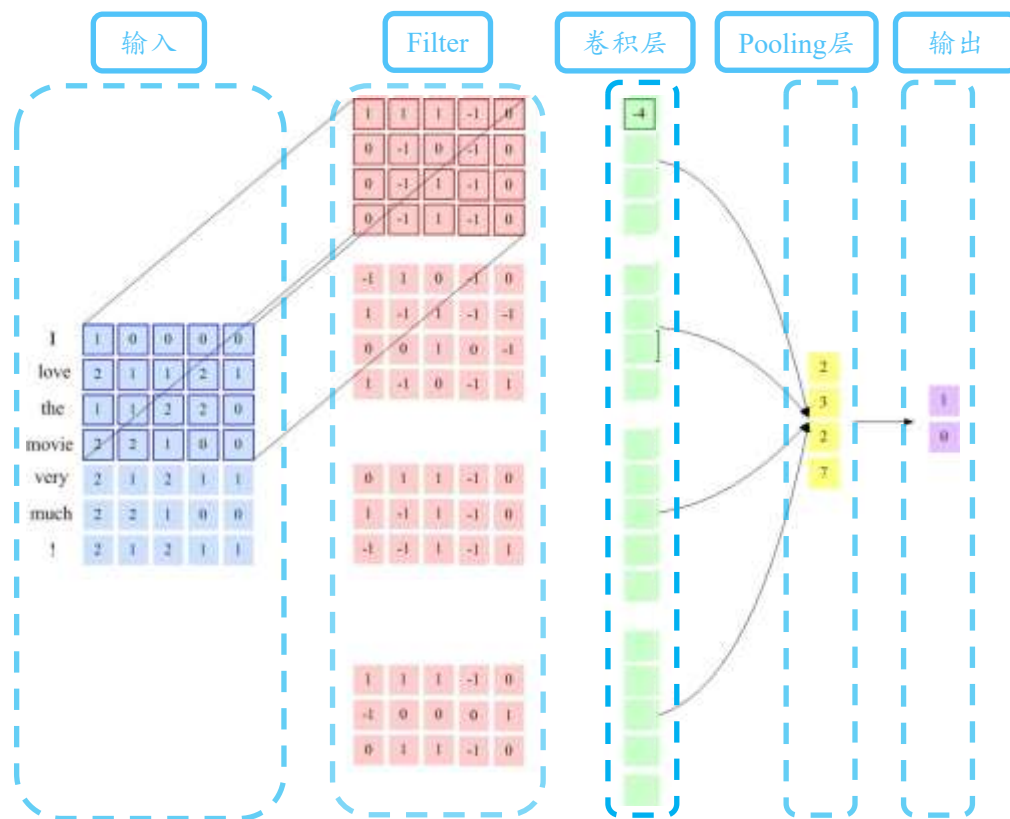
Max p	-0.2	1.6	1.4
-------	------	-----	-----

3	1	2	-3
-1	2	1	-3
1	1	-1	1

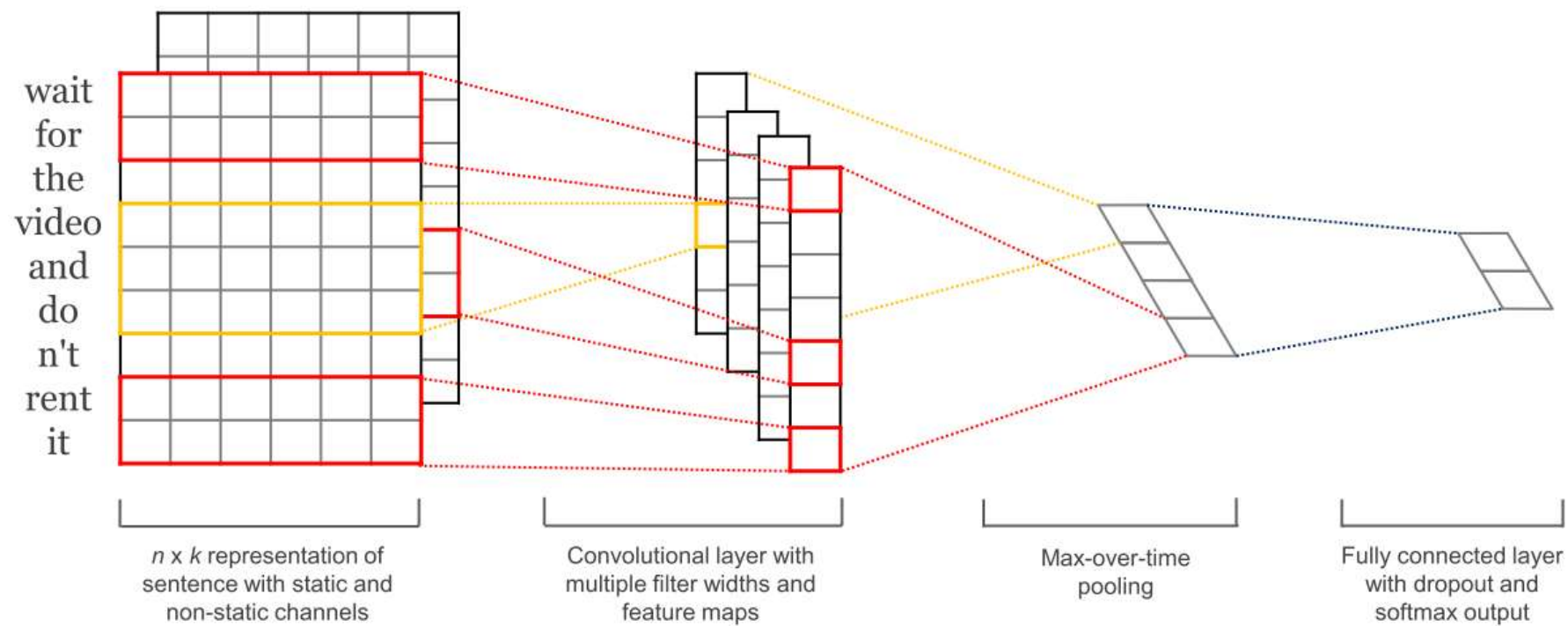
1	0	0	1
1	0	-1	-1
0	1	0	1

1	-1	2	-1
1	0	-1	3
0	2	2	1

文本序列的卷积模型

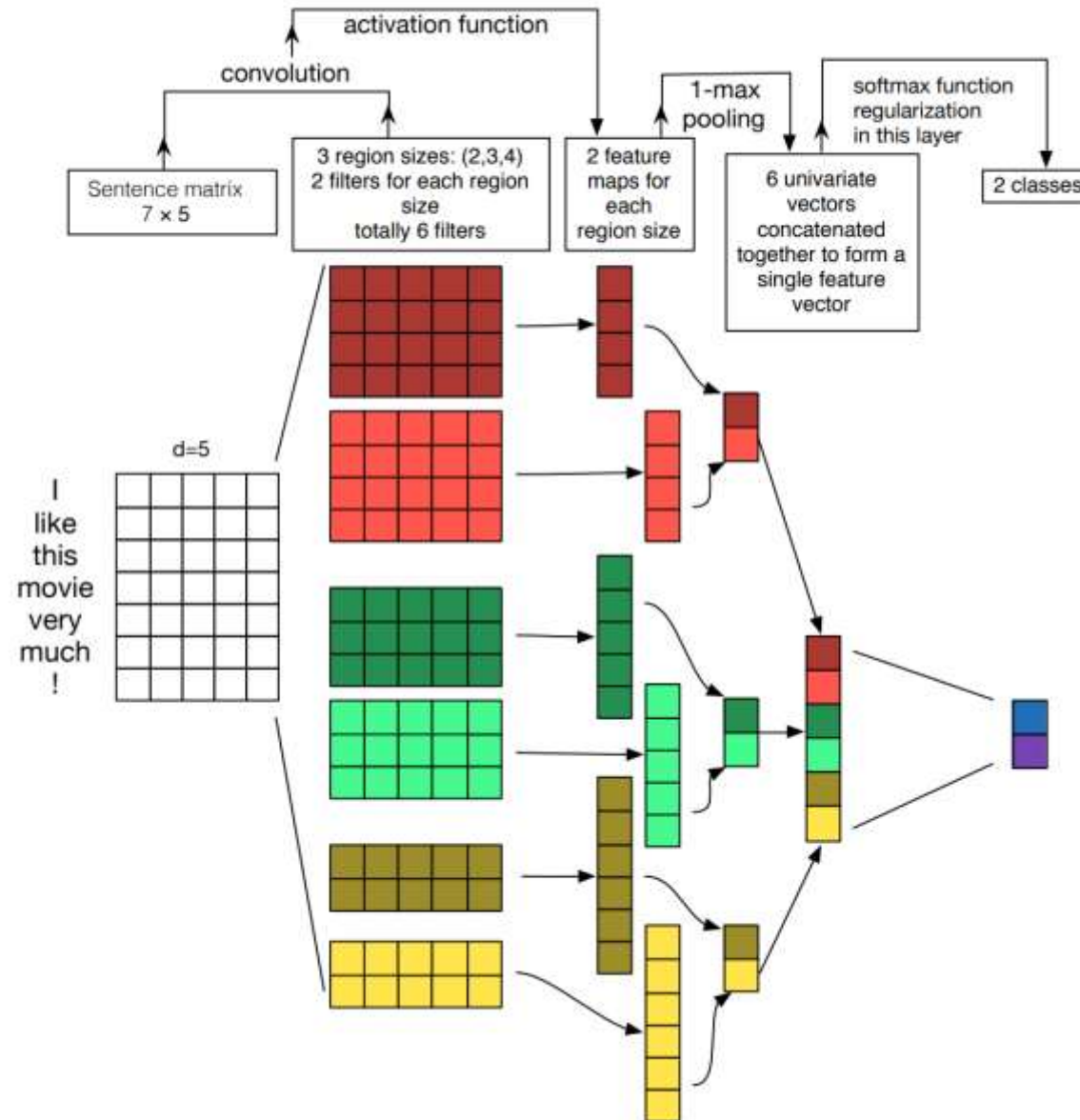


句子分类



Y. Kim. "Convolutional neural networks for sentence classification". In: *arXiv preprint arXiv:1408.5882* (2014).

情感分类



Y. Zhang & B.C. Wallace "A Sensitivity Analysis of (and Practitioners' Guide to) Convolutional Neural Networks for Sentence Classification". In: *arXiv preprint arXiv:1510.03820* (2016).



提问

- 什么是池化？常用的池化操作有哪些？池化的好处是？
- 使用多个卷积核的原因是？
- 卷积核的构造方式是什么样的？



课程目录

Course catalogue

1/ 卷积神经网络概述

2/ 卷积神经网络与自然语言处理

3/ CNN在应用中的超参数选择

CNN在应用中的超参数选择

激活函数

CNN中的卷积核提供的是线性特征，因此，也需要使用激活函数为网络加入非线性。我们在第5章中介绍了几种常见的激活函数，例如sigmoid、tanh、ReLU等。Sigmoid和tanh函数在实际应用中会存在计算速度慢和梯度消失的问题，因此，我们通常会选择ReLU作为激活函数对卷积层的输出进行激活。

CNN在应用中的超参数选择

卷积核的大小和个数

卷积核的大小其合理的值范围在2~5之间。若语料的句子较长，可以考虑使用更大的卷积核。另外，可以在寻找到了最佳的单个卷积核的大小后，尝试在该卷积核的尺寸值附近寻找其他合适值来进行组合。实践证明这样的组合效果往往比单个最佳卷积核表现更出色。

至于卷积核的个数，主要考虑的是当增加卷积核的个数时，训练时间也会加长，因此需要权衡好。当特征图数量增加到将性能降低时，可以加强正则化效果，如将dropout率提高过0.5。

CNN在应用中的超参数选择

Dropout层

当训练一个深度神经网络时，我们可以随机丢弃一部分神经元（同时丢弃其对应的连接边）来避免过拟合，这种方法称为dropout。通常情况下，对于隐藏层的神经元，其dropout率设为0.5时效果最好，这对大部分的网络和任务都比较有效。当dropout率为0.5时，在训练时有一半的神经元被丢弃，只剩余一半的神经元是可以激活的，随机生成的网络结构最具多样性。

CNN在应用中的超参数选择

softmax分类器

softmax函数又称归一化指数函数，它能将一个含任意实数的K维向量Z“压缩”到另一个K维实向量 $\sigma(Z)$ 中，使得每一个元素的范围都在(0,1)之间，并且所有元素的和为1。该函数的形式通常按下面的式子给出：

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

在多分类任务中，类别标签 $y \in \{1, 2, \dots, K\}$ 可以有K个取值，给定一个样本x，softmax函数预测的属于类别k的条件概率为：

$$p(y = k|x) = \text{softmax}(w_k^T x) = \frac{e^{w_k^T x}}{\sum_{k=1}^K e^{w_k^T x}}$$

CNN超参数选择 — 实战速查表

超参数	推荐范围	NLP文本分类实战建议
词嵌入维度	50 / 100 / 200 / 300	小数据集用50-100，大数据集用300
卷积核大小	[3, 4, 5] 组合	多尺度n-gram, Kim(2014) 经典配置
每种核个数	100 ~ 300	通常100即可，更多需加强正则化
Dropout 率	0.3 ~ 0.5	0.5最常用；核数多时可提高至0.6
学习率	1e-4 ~ 1e-3	Adam优化器 + lr=1e-3 是好的起点
Batch Size	32 / 64 / 128	64最常用，GPU显存允许可用128
Epoch数	10 ~ 30	配合Early Stopping, patience=3~5

提示：先用默认值跑通baseline，再逐个调参。改一个参数，固定其余。

词嵌入选择 — 静态 vs 微调 vs 随机

三种词嵌入初始化策略

① 随机初始化 (rand)

Embedding层参数随机初始化，从头训练。适合大规模标注数据。

② 静态预训练 (static)

加载Word2Vec/GloVe预训练向量，训练时冻结不更新。防止小数据过拟合。

③ 微调预训练 (non-static)

加载预训练向量，训练时允许更新。适配当前任务，通常效果最好。

实践建议

数据量 < 5000 条 → 用 static (冻结预训练)，防止过拟合

数据量 > 10000 条 → 用 non-static (微调)，效果通常最优

中文推荐：腾讯AI Lab中文词向量 (200维, 800万词)

学习率与训练策略

学习率调度

Adam $lr=1e-3$ 是最常用的起点

学习率衰减：每 5 个 epoch 乘以 0.5，或 ReduceLR0nPlateau

Warmup：前 1-2 个 epoch 用 $1e-5$ 预热，再切到正常值

Early Stopping (早停法)

监控验证集损失，连续 patience 个 epoch 无改善则停止

推荐 patience = 3~5，保存验证集上最优模型

这是防止过拟合最简单有效的方法

数据增强 (NLP版)

同义词替换：随机替换句中 10% 的词为同义词

随机删除 / 随机交换 / 回译（翻译成英文再翻回中文）

小数据集下可提升 1~3% 准确率

TextCNN 代码实战 — 模型定义 (PyTorch)

```
import torch; import torch.nn as nn
import torch.nn.functional as F
```

```
class TextCNN(nn.Module):
    def __init__(self, vocab, emb_d, n_cls,
                 ks=[3,4,5], n_filter=100):
        super().__init__()
        self.embed = nn.Embedding(vocab, emb_d)
        # 多尺度卷积核 (捕获不同n-gram)
        self.convs = nn.ModuleList([
            nn.Conv1d(emb_d, n_filter, k)
            for k in ks # k=3,4,5
        ])
        self.drop = nn.Dropout(0.5)
        self.fc = nn.Linear(
            n_filter * len(ks), n_cls)
```

```
def forward(self, x): # (B, seq_len)
    e = self.embed(x) # (B, L, D)
    e = e.permute(0,2,1) # (B, D, L)
    # 每个卷积核 + ReLU + MaxPool
    outs = [F.relu(c(e)).max(dim=2)[0]
             for c in self.convs]
    cat = torch.cat(outs, dim=1)
    return self.fc(self.drop(cat))
```

TextCNN 代码实战 — 训练流程

模型初始化

```
model = TextCNN(vocab_size, emb_dim=100,
                n_cls=2, ks=[3,4,5])
optimizer = torch.optim.Adam(
    model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()
```

训练循环

```
best_acc = 0; patience = 0
for epoch in range(30):
    model.train()
    for x, y in train_loader:
        logits = model(x)
        loss = loss_fn(logits, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Early Stopping

```
val_acc = evaluate(model, val_loader)
if val_acc > best_acc:
    best_acc = val_acc; patience = 0
    torch.save(model, 'best.pt')
else:
    patience += 1
    if patience >= 5: break
```

TextCNN 调参实验 — 各超参数影响

卷积核大小的影响 (Zhang & Wallace, 2016)

单一核大小: $k=3$ (unigram-trigram) 通常是最佳单核选择

多核组合 $[3, 4, 5]$ 比任何单核效果好 1~2%

$k > 7$ 通常没有额外增益, 反而增加参数量

特征图数量的影响

50 → 100: 准确率明显提升; 100 → 600: 提升微乎其微

结论: 100~200 是性价比最高的范围

正则化的影响

Dropout 0.5 + L2正则 ($\text{weight_decay}=1e-4$) 是最稳定组合

不用Dropout → 验证集准确率下降 3~5% (过拟合明显)

BatchNorm 在 TextCNN 中效果不如 Dropout

CNN vs RNN — 文本任务如何选择？

对比维度	CNN (TextCNN)	RNN (LSTM/GRU)
擅长捕获	局部 n-gram 模式	长距离依赖关系
训练速度	快（可并行计算）	慢（时序依赖）
适合任务	文本分类、情感分析	机器翻译、文本生成
序列长度	中短句效果好	长文本优势明显
可解释性	高（可视化n-gram权重）	低（隐状态难解释）

实际建议： 文本分类优先试 TextCNN（快且效果好）；
需要理解语序和长距离关系时用 LSTM/GRU；
当前最强方案：预训练 Transformer（BERT），但计算成本高。

现代趋势： 小数据 + 快速推理 → TextCNN 大数据 + 高精度 → BERT/GPT
微调

CNN的局限性与改进方向

TextCNN 的局限

1. 无法捕获长距离依赖：卷积核大小有限，只能看到局部窗口
2. 丢失语序信息：Max Pooling 丢掉了位置信息
3. 对否定词敏感度差："不好" 与 "好" 可能被不同核捕获

改进方向

空洞卷积 (Dilated CNN)：不增加参数量，扩大感受野

深层CNN (DPCNN, 2017)：残差连接 + 深层卷积，接近LSTM

CNN + Attention：池化前加注意力，保留位置信息

CNN + RNN 混合：CNN局部特征 → LSTM序列关系

趋势：TextCNN 仍是工业界文本分类主力（快、易部署、稳定），
但深层语义理解任务已逐步被 BERT 等预训练模型取代。

结束

谢谢